IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 4 November 2024, accepted 14 December 2024, date of publication 20 December 2024, date of current version 2 January 2025.

Digital Object Identifier 10.1109/ACCESS.2024.3520706

RESEARCH ARTICLE

A Hybrid Machine Learning Model for Efficient XML Parsing

MUHAMMAD ALI[1], MINHAJ AHMAD KHAN[1], AND RAIHAN UR RASOOL[2,*]
[1]Department of Computer Science, Bahauddin Zakariya University, Multan 60800, Pakistan
[2]IBM Technologies, Melbourne, VIC 3006, Australia

Corresponding author: Minhaj Ahmad Khan (mik@bzu.edu.pk)

ABSTRACT The Extensible Markup Language (XML) files are extensively used for representing structured data on the web for file configuration, exchanging data between distinct applications, web development, and many other applications. Consequently, effective parsing techniques are necessary for XML files to enhance the performance of applications. The existing parsing techniques have their strengths and weaknesses affecting the performance of applications. Researchers point out that the selection of an efficient and appropriate parser is the most challenging issue regarding a particular condition. This paper proposes a framework XML Parsing Optimization using Hybrid Machine Learning (XPOHML) that makes use of Artificial Neural Network (ANN) and Support Vector Machine (SVM) machine learning techniques for efficient XML parsing. The newly developed framework performs analysis and prediction of different XML parsers using profiling, classification, performance evaluation, and finally generates code for efficient parsing. The XML profiling phase of the XPOHML framework generates a dataset by evaluating the performance of PXTG, SAX, StAX, DOM, and JDOM parsing models on separate cores by applying numerous file sizes. The Classification phase produces the classification model by applying ANN and SVM techniques to identify the appropriate parsing model. The performance evaluation phase of XPOHML assesses the performance of both parsing models through classification metrics (accuracy). Additionally, based on evaluation outcomes, the code generation phase produces an efficient parsing model of XML files. The newly designed and developed XPOHML framework has shown a meaningful improvement in the performance of parsing XML files.

INDEX TERMS Artificial intelligence, artificial neural network, eXtensible markup language, framework, machine learning, model, parsing, support vector machine, XML parsing optimization.

I. INTRODUCTION

Extensible Markup Language, or XML as it is more commonly abbreviated, is a markup language that facilitates data transfer between platforms, including websites and databases. This is possible because XML is cross-platform and allows for incompatibilities between devices and systems. XML is used extensively in online services to store, carry, and transport data over the Internet. It may be used in diverse applications including databases, web applications, e-commerce, multimedia, ERP systems, bioinformatics, and other scientific applications.

In XML, the start tag and end tag, which indicate the beginning and finish of the element, separate the data presented in an XML document. The structure of the document can be defined with the use of XML Schema (XSD) or Document Type Definition (DTD) [1]. XML can be classified as either data-centric or document-centric. A highly structured document is referred to as data-centric and is commonly used for transferring data from a single platform to another. Businesses typically include information on sales orders, stock quotes, travel schedules, and scientific data analysis are a few examples of data-centric. Document-centric XML is another category, in addition to data-centric XML. There is a

The associate editor coordinating the review of this manuscript and approving it for publication was Yilun Shang.

* Raihan ur Rasool is an IBM Australia's employee; however, this research contains his personal views and not of IBM.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

lot of text in this kind of XML, which has loose structures. Examples include product catalogs, legal documents, and news sources like CNN RSS Feeds.

The efficiency of XML parsing depends upon the execution of diverse XML operations including verification of well-formedness, traversal of tags along with metadata, and construction of a parse tree. To speed up XML parsing, a variety of hardware, software, and hybrid techniques are employed [2]. The hardware-based methods make use of specialized hardware accelerators like LTMAdder [3] and FPGA custom circuits [4]. In contrast, the software-based approaches include methods like pull-based parsing [5], lazy parsing [6], and schema-specific parsing [7]. In the same way, both hardware and software parsing techniques are used in the hybrid XML parsing approaches [8]. To enhance XML parsing, serial and parallel approaches have been extensively researched and incorporated into current practices.

The applications of Artificial Intelligence (AI) in research, engineering, and industry have grown rapidly in the last several years [9]. The various machine learning techniques or algorithms that learn from data and automatically improve performance for a wide range of applications, including XML parsing, reverse engineering, fault diagnosis, advanced modeling, predictive maintenance, quality control, control strategies, and system or component optimization [10], [11], [12], [13], [14], [15]. AI generally combines human intelligence and behaviour into computers or systems [16], whereas machine learning (ML) automates the process of developing models by training from experience or data [17]. Examples, such as the data used for learning and evaluation, are utilized as guidelines in the learning stage of machine learning algorithms. The training set, validation set, and test set comprise the data set. First, the learning algorithm is trained on the training set. Next, its parameters are adjusted on the validation set. Lastly, the learning algorithm's performance is assessed on the test set [18]. Building a model that learns a function to explain a particular pattern in a dataset while another unknown pattern exists is the key objective of machine learning. Machine learning algorithms can be classified into three categories [9], [19], [20] known as supervised learning, supervised learning, and reinforcement learning.

In this paper, we propose a framework XML Parsing Optimization using Hybrid Machine Learning (XPOHML) that incorporates techniques of ANN and SVM to analyze and predict the performance of different parsers through XML profiling, classification, performance evaluation, and code generation for efficient parsing. The XML profiling step of the XPOHML framework creates a dataset by assessing the effectiveness of the PXTG, SAX, StAX, DOM, and JDOM parsing models on various cores using a range of file sizes. The classification phase produces the prediction model by applying ANN and SVM techniques to identify the appropriate algorithm. The XPOHML framework then evaluates performance to assess these algorithms' efficacy. Ultimately, the code generation phase generates the final code for effectively parsing XML files based on previous assessment. The remaining document has the following structure. Section II describes the related work, in terms of XML parsing, ANN, and SVM architectures. The proposed XPOHML framework used to analyze and predict performance through XML profiling, classification, performance evaluation, and code generation for efficient parsing is presented in Section III. Section IV presents the experimental setup and findings achieved by implementing the proposed framework and the profiling of existing widely known parsing models. The conclusion and future work are deliberated in the final Section of this document.

II. RELATED WORK

To enhance the parsing of XML documents, a variety of methods have been widely designed and executed by researchers. The Document Object Model (DOM) [21] parsers return an object to control elements after storing a complete hierarchical document in memory after parsing. This leads to an inefficient parsing for large documents. Another Java-based parser is Simple API for XML (SAX) [22] parses XML documents in a top-down manner with no random or direct access. Similarly, the Streaming API for XML (StAX) uses a pull-based method for extracting data. On the other hand, JDOM is a Java-based library that has a small memory footprint and is used for processing XML files as fast as SAX but only for small documents. The Parallel XML Tree Generator (PXTG) algorithm [23] is designed for efficient parsing. It takes an XML file as input and produces n subtrees. This approach serially traverses the input file in n parts before creating parallel subtrees. The traversal is determined by different number of characters in the XML file. In order to bring in a proper format for XML traversal, the XML contents are patched as merged or replaced. The contents of XML files are tuned by placing duplicate entries for nodes and edges to achieving the hierarchy and the well-formedness of each part. Additionally, the well-formed chunks are sent to individual cores. This process is applied to create parallel pre-parsed subtrees. Lastly, PXTG produces well-formed subtrees. VTD-XML is a novel non-extracted XML parsing technique that Zhang introduced [24]. By analyzing the XML document, VTD-XML generated a one-dimensional array that could reduce the amount of memory required. Additionally, a variety of formats, such as binary XML [25], have been suggested in the literature to enhance the performance of XML parsing.

The requirement for strong predictive models is becoming more and more demanding, because it is necessary to perform predictions for many daily life scenarios, such as changes in stock prices, weather patterns, or the development of features in complicated industrial processes. Numerous techniques were employed for this reason, including mathematical/empirical-based equations, regression, and statistical-based equations. Recently, various techniques of soft computing, such as, gene expression programming (GEP), SVM, the adaptive neuro-fuzzy inference system (ANFIS), and ANN

were also utilized to predict. To find the best XML parsing algorithm, our proposed framework XPOHML incorporates SVM with kernel and ANN with a hidden layer.

In this Section, we discuss the extensive usage of ANN and SVM in the fields of system identification and predictive control [26], [27]. Even though ANNs produce good results, there are still significant design challenges. These include determining how many neurons to include in hidden layers, the issue of overfitting, and the presence of local minima in the target function. SVM are considered incredibly effective regression and classification tools. Among their many benefits are their strong generalization skills and optimization method, which is based on a convex function without a local minimum [28]. According to its flexibility to address challenges in pattern recognition [29], including non-linearity and multiple dimensions, these models are easy to use and optimized. SVM finds extensive application in face recognition [30], image processing [31], speech recognition [32], face detection [33], and related domains.

The most applicable instruments for resolving complex, contemporary issues are the hybridized classification techniques of optimization algorithms using ANN and SVM. Sonar targets [34], [35], [36], [37], [38], [39], underwater acoustical targets [40], [41], medical image categorization [42], [43], and brain [44], [45] or breast tumour diagnostics [46], [47], [48] are a few examples. In contrast, our approach builds a framework that combines ANN and SVM methodologies for analyzing and predicting the performance of various parsers through XML profiling, classification, performance evaluation, and code generation for effective parsing.

III. PROPOSED METHODOLOGY

This paper proposes an XPOHML framework that analyses the performance of XML parsing, predicts the efficient parsing model using ANN and SVM techniques, evaluates performance, and generates code for an optimized XML parser. The XPOHML framework incorporates classification models using ANN and SVM while configuring different parameters. The architecture of the XPOHML framework is stated in Figure 1. The first phase of the proposed framework calculates the parsing time of various algorithms including StAX, SAX, DOM, JDOM, PXTG1, PXTG4, PXTG8, PXTG12, and PXTG16. This phase ends by generating the dataset by considering the parsing time for these algorithms. PXTG1 handles XML files as a single chunk, while PXTG4, PXTG8, PXTG12, and PXTG16 segment XML files into 4, 8, 12, and 16 parts respectively. The XML profiling step utilizes various XML file sizes to calculate the parsing times of these algorithms. The next phase of the XPOHML framework performs classification by applying two machine learning techniques ANN and SVM. After the configuration of different parameters, ANN and SVM create a classification model for prediction. The next phase evaluates the performance of classifiers by applying classification metrics. The final phase generates code that invokes the most efficient parsing algorithm in terms of execution time.

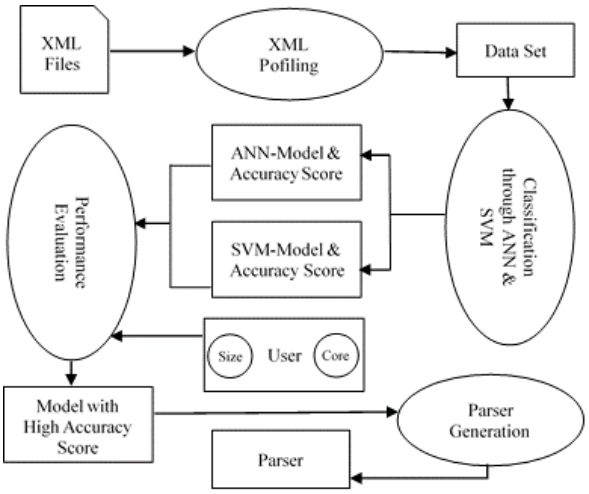


FIGURE 1. Proposed framework XPOHML architecture.

Figure 1 describes the four major components that constitute the XPOHML framework. The initial phase of this framework is XML profiling, which is used to measure the parsing times of various algorithms to produce a dataset. The second phase performs classification by applying two machine learning techniques ANN and SVM to analyze the performance of parsers and produce classifiers for ANN and SVM. This step results in the creation of classification models. The third phase of the proposed framework is to find the best accuracy score by applying classification metrics. The fourth and final phase parser generator uses user-provided parameters for size and cores to predict the most effective algorithm.

Algorithm 1 XPOHML (XML Files)

Let η be the learning rate
Let **ep** be the number of epochs

1. Read XML Files
   //XML Profiling
2. For each core:
   Call StAX|SAX|DOM|JDOM|PXTG1|PXTG4|PXTG8| PXTG12|PXTG16
   Create dataset **D**
3. Performs pre-processing on the **D**
4. ANN_Model=ANN_ModelGenerator(η, **ep**, **D**)
5. SVM_Model=SVM_ModelGenerator(η, **ep**, **D**)
6. Input **Size**, **Core**
7. ParserCodeGenerator(**Size**, **Core**)

Step 1 of ***XPOHML*** (Algorithm 1) receives XML files of different sizes as input. Step 2 sends XML files on the separate core for parallel XML parsing. Each core receives XML files and calls the appropriate parser to measure the parsing time. The parsing time generated by each parser creates a dataset. The occurrences of this dataset are established on file size (Size), number of cores (Core), and parsing time (Performance) of each algorithm. Step 3. performs pre-processing on the dataset by finding the name of the most efficient

algorithm against each instance. Finally, the instances of the dataset used by ANN and SVM for the classification are "Size", "Core", and "Efficient_Algo". Steps 4 and 5 call Algorithm 2 and Algorithm 3 respectively for creating the classification model. Eventually, the XPOHML framework creates two classification models for predicting the most efficient algorithm. Step 6 allows the user to enter size and core, against the user wants to generate the most efficient parser. The last step of Algorithm 1 generates an efficient parser.

A. XML PROFILING

XML Profiling is the first phase of the XPOHML framework that gathers data and produces dataset D. The performance of several algorithms, such as SAX, StAX, DOM, JDOM, PXTG, and the PXTG variations, is measured using this procedure, as shown in Figure 2. XML files are handled as a single chunk by PXTG1, however, these are divided into 4, 8, 12, and 16 chunks by PXTG4, PXTG8, PXTG12, and PXTG16, respectively for parallel processing.

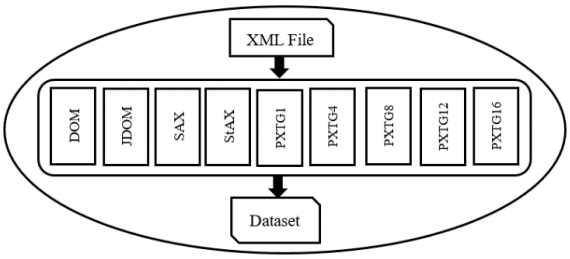


FIGURE 2. XML profiling.

All parsing models including the variants of the PXTG parsing model are applied on various XML file sizes 1, 2, 3, 4, 5, 10, 15, 20, 25, 50, 75, 100, and 125 MB by using 1, 2, 4, 6, 8, 10, 12, 14, and 16 cores. The objective of the XML profiling phase is to generate the dataset D for the next phase. The results of XML profiling are given in the result section. A minor modification is required in a dataset to become useable for ANN and SVM techniques. The parsing time of all algorithms is compared and replaced by the name of the most efficient algorithm in terms of parsing time for each particular instance. After the modification of the dataset, the instances of a finalized dataset will be labeled as file size (Size), number of cores (Core), and name of the optimized algorithm (Efficient_Algo).

B. CLASSIFICATION

The classification phase of the proposed framework is to produce a classification model. The growing understanding of the composition and operation has produced several classification techniques over the past decade. The present study follows ANN and SVM techniques for classification and prediction-making as described below.

1) CLASSIFICATION USING ANN

Applications for neural networks (ANN) include data analysis, pattern recognition, and categorization. The primary structure of ANN consists of an input layer, one or more hidden layers, and one output layer. Many neurons make up each layer, and the connections between these neurons are assigned varying weights. In the input layer, each neuron receives a single data source. The output of every input layer serves as a source of input for every hidden layer neuron. During the training phase, each neuron's weight is determined. Any novel input vector is fed into the input layer of the neural network once it has been trained. The value of each node is determined by adding all the values that enter the node, and multiplying the value of the input node by the connection weight. The architecture of ANN used by the XPOHML framework is given in Figure 3.

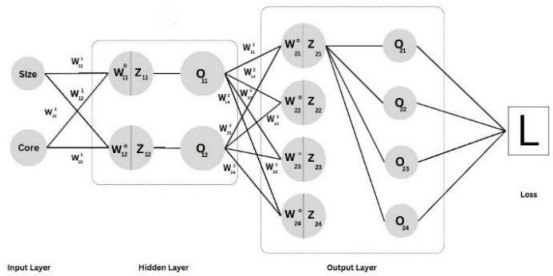


FIGURE 3. Architecture of ANN used by proposed framework XPOHML.

The ANN architecture of the proposed framework XPOHML is based on an input layer, one hidden layer, and an output layer. The forward propagation process is initiated to create a classification model or classifier by selecting the input features size and core. The input is given to the hidden layer with associating weights and biases by applying (1) to calculate Z_{1i}. Then the input layer performs an activation function sigmoid for Z_{1i} and produces O_{1i}, by applying (2).

$$Z_{1i} = Size \cdot W^1_{1i} + Core \cdot W^1_{2i} + W^0_{1i} \tag{1}$$

$$O_{1i} = sigmoid\,(Z_{1i}) = 1/\left(1 + e^{Z_{1i}}\right) \tag{2}$$

where $i = 1, 2$.

Then, the forward propagation process is further applied on hidden layers to produce an output layer using (3).

$$Z_{2i} = O_{11} \cdot W^2_{1i} + O_{12} \cdot W^2_{2i} + W^0_{2i} \tag{3}$$

where $i = 1, 2, 3, 4$.

The value provided by the sigmoid activation function ranges from 0 to 1. It is pertinent to mention that while dealing with multi-class classification problems, the sigmoid activation function is not recommended. The softmax activation function is utilized to compute the relative probabilities. The probability of each class is also returned by the softmax function. The inputs are expressed as a discrete probability distribution using the softmax function by using (4).

$$O_{2i} = \text{softmax}\,(Z_{2i}) = \frac{e^{Z_{2i}}}{e^{Z_{21}} + e^{Z_{22}} + e^{Z_{23}} + e^{Z_{24}}} \tag{4}$$

where $i = 1, 2, 3, 4$.

Loss functions, which are typically utilized during classification processes, quantify possible errors in the model as well as the difference between expected and actual values. One of these functions, used for multi-class classification is categorical-cross-entropy or softmax loss. Entropy in simple terms is a measure of how uncertain an occurrence is throughout training. However, different weight adjustments of models are made via cross-entropy loss. The loss function measures prediction errors in the model to direct the learning process. Accuracy characteristic quantifies loss function. In machine learning, classification is essential as it forms the basis for numerous applications, such as image recognition, algorithm recognition, and email categorization. The goal is to reduce the loss with the notion that a better model has a lesser loss. A softmax activation and a categorical cross-entropy loss are employed for multiclass classification. The softmax activation is applied to the raw outputs of the neural network in a multiclass classification problem. This results in a vector of predicted probabilities across the input classes. After calculating loss, the gradient is calculated between the output and hidden layer to update weights and biases using (5) and (6) respectively.

$$\frac{\partial L}{\partial W_{11}^2} = \frac{\partial L}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{11}^2} + \frac{\partial L}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{11}^2} + \frac{\partial L}{\partial O_{23}} \cdot \frac{\partial O_{23}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{11}^2} + \frac{\partial L}{\partial O_{24}} \cdot \frac{\partial O_{24}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{11}^2} \tag{5}$$

$$\frac{\partial L}{\partial W_{21}^0} = \frac{\partial L}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{21}^0} + \frac{\partial L}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{21}^0} + \frac{\partial L}{\partial O_{23}} \cdot \frac{\partial O_{23}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{21}^0} + \frac{\partial L}{\partial O_{24}} \cdot \frac{\partial O_{24}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial W_{21}^0} \tag{6}$$

where,

$$\frac{\partial L}{\partial O_{21}} = \frac{\partial}{\partial O_{21}}[\sum_{i=1}^{4} -y_i log(O_{2i})] = -\frac{y_1}{O_{21}} \tag{7}$$

$$\frac{\partial L}{\partial O_{22}}, \frac{\partial L}{\partial O_{23}}, \frac{\partial L}{\partial O_{24}} = -\frac{y_2}{O_{22}}, -\frac{y_3}{O_{23}}, -\frac{y_4}{O_{24}} \tag{8}$$

$$\begin{aligned}\frac{\partial O_{21}}{\partial Z_{21}} &= \frac{\partial}{\partial Z_{21}}\left[\frac{e^{Z_{21}}}{\sum_{i=1}^{4} e^{Z_{2i}}}\right] \\ &= \frac{\left[\sum_{i=1}^{4} e^{Z_{2i}}\right] \frac{\partial}{\partial Z_{21}} e^{Z_{21}} - e^{Z_{21}} \frac{\partial}{\partial Z_{21}}\left[\sum_{i=1}^{4} e^{Z_{2i}}\right]}{\left[\sum_{i=1}^{h} e^{Z_{2i}}\right]^2} \\ &= O_{21}(1 - O_{21})\end{aligned} \tag{9}$$

$$\frac{\partial O_{22}}{\partial Z_{21}} = \frac{\partial}{\partial Z_{21}}\left[\frac{e^{Z_{22}}}{\sum_{i=1}^{4} e^{Z_{2i}}}\right] = -O_{22}(O_{21}) \tag{10}$$

$$\frac{\partial O_{23}}{\partial Z_{21}}, \frac{\partial O_{24}}{\partial Z_{21}} = -O_{23}(O_{21}), -O_{24}(O_{21}) \tag{11}$$

$$\frac{\partial Z_{21}}{\partial W_{11}^2} = \frac{\partial}{\partial W_{11}^2}\left[W_{11}^2 \cdot O_{11} + W_{21}^0\right] = O_{11} \tag{12}$$

$$\frac{\partial Z_{21}}{\partial W_{21}^0} = \frac{\partial}{\partial W_{21}^0}\left[W_{11}^2 \cdot O_{11} + W_{21}^0\right] = 1 \tag{13}$$

Finally, putting the values from (7)-(13) in (5) and (6), will generate the following equations.

$$\frac{\partial L}{\partial W_{11}^2} = -\frac{y_1}{O_{21}} \cdot O_{21}(1 - O_{21}) \cdot O_{11} + \frac{y_2}{O_{22}} \cdot O_{22}(O_{21}) \cdot O_{11} + \frac{y_3}{O_{23}} \cdot O_{23}(O_{21}) \cdot O_{11} + \frac{y_4}{O_{24}} O_{24}(O_{21}) \cdot O_{11} \tag{14}$$

$$\frac{\partial L}{\partial W_{21}^0} = -\frac{y_1}{O_{21}} \cdot O_{21}(1 - O_{21}) + \frac{y_2}{O_{22}} \cdot O_{22}(O_{21}) + \frac{y_3}{O_{23}} \cdot O_{23}(O_{21}) + \frac{y_4}{O_{24}} \cdot O_{24}(O_{21}) \tag{15}$$

Equations (16) and (17) are used to calculate weights and biases between the output and input layer.

$$\begin{aligned}\frac{\partial L}{\partial W_{11}^1} &= \frac{\partial L}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^1} \\ &+ \frac{\partial L}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^1} \\ &+ \frac{\partial L}{\partial O_{23}} \cdot \frac{\partial O_{23}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^1} \\ &+ \frac{\partial L}{\partial O_{24}} \cdot \frac{\partial O_{24}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^1}\end{aligned} \tag{16}$$

$$\begin{aligned}\frac{\partial L}{\partial W_{11}^0} &= \frac{\partial L}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^0} \\ &+ \frac{\partial L}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^0} \\ &+ \frac{\partial L}{\partial O_{23}} \cdot \frac{\partial O_{23}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^0} \\ &+ \frac{\partial L}{\partial O_{24}} \cdot \frac{\partial O_{24}}{\partial Z_{21}} \cdot \frac{\partial Z_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial Z_{11}} \cdot \frac{\partial Z_{11}}{\partial W_{11}^0}\end{aligned} \tag{17}$$

where,

$$\frac{\partial Z_{21}}{\partial O_{11}} = \frac{\partial}{\partial O_{11}}\left[W_{11}^2 \cdot O_{11} + W_{21}^2 \cdot O_{12} + W_{21}^0\right] = W_{11}^2 \tag{18}$$

$$\begin{aligned}\frac{\partial O_{11}}{\partial Z_{11}} &= \frac{\partial}{\partial Z_{11}}[\sigma(Z_{11})] = \sigma(Z_{11})[1 - \sigma(Z_{11})] \\ &= O_{11}(1 - O_{11})\end{aligned} \tag{19}$$

$$\frac{\partial Z_{11}}{\partial W_{11}^1} = \frac{\partial}{\partial W_{11}^1}\left[W_{11}^1 \cdot Size + W_{21}^1 \cdot Core + W_{11}^0\right] = Size \tag{20}$$

$$\frac{\partial Z_{11}}{\partial W_{11}^0} = \frac{\partial}{\partial W_{11}^0}\left[W_{11}^1 \cdot Size + W_{21}^1 \cdot Core + W_{11}^0\right] = 1 \tag{21}$$

Finally, putting the values from (7)-(15), and from (18)-(21) in (16) and (17), will generate the following equations:

$$\frac{\partial L}{\partial W_{11}^1} = -\frac{y_1}{O_{21}} \cdot O_{21}(1 - O_{21}) \cdot W_{11}^2 \cdot O_{11}(1 - O_{11}) \cdot Size + \frac{y_2}{O_{22}} \cdot O_{22}(O_{21}) \cdot W_{11}^2 \cdot O_{11}(1 - O_{11}) \cdot Size$$

$$+\frac{y_3}{O_{23}}\cdot O_{23}\,(O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11})\cdot Size$$
$$+\frac{y_2}{O_{24}}\cdot O_{24}(O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11})\cdot Size \tag{22}$$

$$\frac{\partial L}{\partial W_{11}^0}=-\frac{y_1}{O_{21}}\cdot O_{21}\,(1-O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11})$$
$$+\frac{y_2}{O_{22}}\cdot O_{22}\,(O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11})$$
$$+\frac{y_3}{O_{23}}\cdot O_{23}\,(O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11})$$
$$+\frac{y_2}{O_{24}}\cdot O_{24}(O_{21})\cdot W_{11}^2\cdot O_{11}\,(1-O_{11}) \tag{23}$$

The optimization of machine learning is regarded as the process of iteratively increasing the accuracy of a machine learning model while decreasing its error rate. Machine learning models can generalize and predict unseen data based on knowledge gained from training data. Reducing the amount of error between the true and anticipated outputs is a key objective of machine learning algorithm training. A loss or cost function, which is usually provides the discrepancy between the expected and actual value of data for optimization. Machine learning models aim to minimize this loss function or reduce the difference between actual and predicted data. Through iterative optimization, the machine learning model will progressively improve its ability to classify data and predict outcomes. The following equation is a fundamental optimization model for regular gradient descent:

$$W_t = W_{t-1} - \eta\frac{\partial L}{\partial W_{t-1}} \tag{24}$$

W_t refers to the weights at current time, while W_{t-1} shows previous time, η represents the learning rate, and the partial derivative of loss function regarding the weight (dW) and bias (db) at the previous step are all represented in this equation. The momentum-based parameter update rate appears in the manner given below:

$$W, b = W - \eta M_{dW}, b - \eta M_{db} \tag{25}$$

The following equations define M_{dW} and M_{db}

$$M_{dW} = \beta M_{dW_{pre}} + (1-\beta)dW \tag{26}$$
$$M_{db} = \beta M_{db_{pre}} + (1-\beta)db \tag{27}$$

whereas, RMSprop updating functions as follows:

$$W, b = W - \eta * \frac{dW}{\sqrt{R_{dW}+\varepsilon}}, b - \eta * \frac{db}{\sqrt{R_{db}+\varepsilon}} \tag{28}$$

R_{dW} and R_{db} are defined as:

$$R_{dW} = \beta_1 * R_{dw_{pre}} + (1-\beta_1)\,dW^2 \tag{29}$$
$$R_{db} = \beta_2 * R_{db_{pre}} + (1-\beta_2)db^2 \tag{30}$$

The combination of the momentum-based and RMSprop results in an ADAM optimizer:

$$W, b = W - \eta * \frac{M_{dW}}{\sqrt{R_{dW}+\varepsilon}}, b - \eta * \frac{M_{db}}{\sqrt{R_{db}+\varepsilon}} \tag{31}$$

Algorithm 2 ANN_ModelGenerator ($\boldsymbol{\eta}$, ep, D)

1. Initialize biases $W_{[][]}^0$ *and* weights $W_{[][]}^1$, $W_{[][]}^2$.
2. **X**= **D**[Size, Core]
3. **Y** = **D**[Efficient_Algo]
4. FOR **e**=1 to **ep**
   i. Perform forward propagation from the input layer to the hidden layer using (1).
   ii. Apply the sigmoid activation function to produce intermediate output using (2).
   iii. Initiate forward propagation from the hidden layer to the output layer using (3).
   iv. Apply the softmax activation function to produce the final output using (4).
   v. Calculate loss using categorical_cross_entropy.
   vi. Compute gradients for weights and biases from the output layer to the hidden layer using (14)-(15).
   vii. Compute gradients for weights and biases from the output layer to the input layer using (22)-(23).
   viii. Update parameters using (31).
   END FOR
5. Return ANN_Model

β_1 is standardized and set to 0.9, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$.

ANN_ModelGenerator (Algorithm 2) receives learning rate ($\boldsymbol{\eta}$), number of epochs (**ep**), and dataset (**D**). Step 1 initializes W^0, W^1, and, W^2 with random values. W^1 represents weights within the input and hidden layers, W^2 represents weights within the hidden and output layers, and W^0 represents biases. Steps 2 and 3 extract the input features and target feature in X and Y respectively. Step 4 consists of eight sub-steps. Sub-step 1 computes Z_{1i} by initiating the forward propagation from the input layer to the hidden layer using (1). Sub-step 2 performs the activation function sigmoid on Z_{1i} to produce O_{1i} as given in (2). Sub-step 3 computes Z_{2i} by performing the forward propagation from hidden layer to output layer using (3). Sub-step 4 performs the activation function softmax on Z_{2i} to produce O_{2i} as given in (4). Phase 5 measures loss by applying categorical_cross_entropy. Sub-steps 6 and 7 compute gradients for weights and biases between input and output layers. Last phase updates weights and biases to reduce loss. Step 4 is repeated up to the number of epochs to improve the performance of the ANN model. The final step returns the classifier to Algorithm 1.

2) CLASSIFICATION USING SVM

The other machine learning technique used by the XPOHML framework is SVM. This technique initially targeted classification issues, but it was quickly expanded to include regression and other learning challenges. SVM algorithm-based models are distinguished by their superior capacity for generalization. This is because SVM outperforms other traditional forecasting methods in terms of generalization since it can minimize structural risk, namely the top limit of the generalization error, rather than empirical error. SVM generates a

low-dimensional, nonlinear regression. A high dimensional feature is created by nonlinearly mapping the input data, and a linear regression is then performed in this modified space. A Support Vector Classifier (SVC) specifically builds a hyperplane or collection of high-dimensional hyperplanes that divide training data into several classes and then utilize the hyperplanes to categorize an image (or set of images) appropriately. Developed in 1963 by Vapnik and Lerner [49], the initial SVC algorithm was a linear classifier that yielded no results if no hyperplane could be located to fully differentiate the training set. More flexibility in training data separation was proposed by Boser et al. [50] in 1992 by utilizing the so-called "Kernels" to embed nonlinearity in SVC. The employment of kernel functions allows SVC to function in a high-dimensional feature space without converting. All that is required is the computation of the inner product between each pair of data in the space [51]. Complete separation of the training data is not always achievable, even with Kernels. The most widely used mapping software, such as ENVI [52] and EnMAP [53], uses the current standard SVM technique for categorization of remote sensing data (C-SVC), which was designed in 1995 by Cortes and Vapnik [54]. Even if C-SVC is unable to fully separate the training data, it can still classify entities. The terms "hard margin" and "soft margin" classifiers, respectively, refer to SVC and C-SVC. When a classifier uses a hard margin, it must fully segregate the training set in order to produce a result. A soft margin classifier, on the other hand, selects the most effective hyperplanes to divide the training data while permitting some inaccuracy in the separation of the training data. In this sense, the soft margin classifiers always yield a result, but they are highly dependent on the model's parameterization and training set.

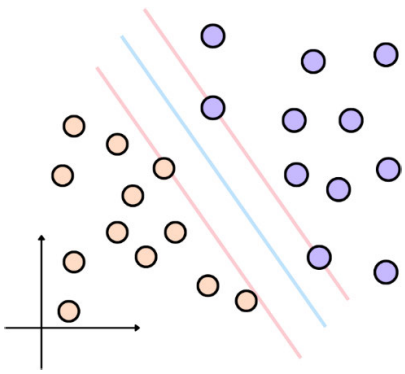


FIGURE 4. SVM hard-margin.

Let the pairs of training features and class labels be $(x_1, y_1), (x_2, y_2), \ldots (x_n, y_n)$. Here, x is the collection of algorithm training features, and y denotes the corresponding class labels. Y_i may have a value of $+1$ or -1, depending on the class. An endless number of hyperplanes can be drawn to divide two classes. However, the hyperplane needs to be drawn with the biggest margin possible between the dividing planes. The weights W^i can be changed to divide the data into two classes. The margins are defined by the hyperplanes, and

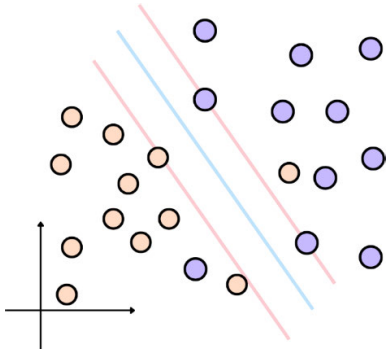


FIGURE 5. SVM soft-margin.

it can be expressed as.

$$H_1\colon W^0X_0 + W^1X_1 + W^2X_2 = 1, \quad forY_i = +1 \quad \forall i = 1, 2, \ldots, n. \tag{32}$$

$$H_2\colon W^0X_0 + W^1X_1 + W^2X_2 = -1, \quad forY_i = -1 \quad \forall i = 1, 2, \ldots, n. \tag{33}$$

Accordingly, any algorithm that appears on or above H_1 has its features set to class $+1$, while any algorithm that appears on or below H_2 has its features set to class -1. From (32) and (33), we obtained (34).

$$Y_i(W^0X_0 + W^1X_1 + W^2X_2) > 1 \quad \forall i = 1, 2, \ldots, n. \tag{34}$$

The margin width is therefore set to $2/\|W\|$. Therefore, minimizing the following term and maximizing the margin are comparable.

$$\begin{aligned} &\min(0.5\,\|W\|^2) \\ &\text{subject to } Y_i(W^0X_0 + W^1X_1 + W^2X_2) > 1, \\ &\qquad \forall i = 1, 2, \ldots, n. \end{aligned} \tag{35}$$

Support vectors are training features that are located on hyperplanes H_1 or H_2 having the following equation.

$$W^0X_0 + W^1X_1 + W^2X_2 = 0, \forall i = 1, 2, \ldots, n. \tag{36}$$

On the other side, following method of evaluation is applied for convex optimization problem:

$$\begin{aligned} &Min_{w,w^0,\xi}\,\boldsymbol{\tau}\left(w, w^0, \xi\right) = 0.5 \cdot W^T W + C \sum\nolimits_{i=1}^{n} \xi_i, \\ &subjectto\ Y_i\left[W^T \varphi\left(X_i\right) + W^0\right] \geq 1 - \xi_i, \\ &\qquad \xi_i \geq 0, \quad \forall i = 1, 2, \ldots, n. \end{aligned} \tag{37}$$

The slack variables, represented by ξ_i, help allow misclassifications within the set of inequalities as shown in Figure 5. The objective function's first component seeks to maximize the feature space margin between the two classes, while the second component reduces the misclassification error. The algorithm's tuning parameter should be the positive real constant C [55], whereas φ is a non-linear function. Since a direct optimization problem would be extremely difficult, Lagrange

multipliers are used to solve this limited optimization problem [56]. The cost function is expressed as follows.

$$L = 0.5 \cdot \|W\|^2 - \sum_{i=1}^{n} \alpha_i \{Y_i \left[W^T \varphi(X_i) + W^0 \right] - 1\} \tag{38}$$

where α_i, $i = 1, 2, \ldots, n$ are Lagrangian multipliers with following constrains $\alpha_i > 0$. Because saddle points are obtained by maximizing L with respect to a and minimizing L with respect to W and W^0, there should be a minus sign before Lagrangian multipliers [57]. Next, zero is assigned to the derivatives of the cost function with respect to W and W^0.

$$\frac{\partial L}{\partial W} = 0 \rightarrow W = \sum_{i=1}^{n} \alpha_i Y_i X_i \tag{39}$$

$$\frac{\partial L}{\partial W^0} = 0 \rightarrow \sum_{i=1}^{n} \alpha_i Y_i = 0 \tag{40}$$

The maximum margin problem has a dual representation when (39) and (40) are substituted in (38) [56]. The following equation can be used to solve the optimization problem and create an ideal hyperplane with the maximum geometric margin.

$$\tilde{L}(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j Y_i Y_j K(X_i, X_j) \tag{41}$$

where $K(X_i, X_j) = X_i^T X_j$ is known as the kernel function. Nonlinear SVM, however, is employed if linear data separation is not achievable.

For data that cannot be linearly separable, several kernels can be employed to find a separating hyperplane. Memory management, task management, and disc management are all handled by the kernel. The "data" is fed into the kernel, which changes it into the format required for data processing. Higher dimensions and nonlinearity are dealt with using kernels. There are several different types of kernels used in machine learning techniques, including Gaussian, sigmoid, polynomial, RBF, and linear.

By calculating the dot product of two feature vectors, the linear kernel effectively determines how similar the two feature vectors were in the original feature space, which is defined as follows:

$$K(X_i, X_j) = X_i^T X_j + c \tag{42}$$

For problems where all of the training data has been standardized, polynomial kernels work best. Three variables can be altered: the polynomial degree (p), the constant term (c), and the slope (alpha).

$$K(X_i, X_j) = (\alpha {X_i}^T X_j + c)^p \tag{43}$$

The Hyperbolic Tangent Kernel is commonly referred as the Sigmoid Kernel or the Multilayer Perceptron (MLP) kernel. The bipolar sigmoid function, which is widely used as a synthetic neuron activation function in the field of Neural Networks, is the source of the sigmoid kernel.

$$K(X_i, X_j) = tanh(\alpha {X_i}^T X_y + c) \tag{44}$$

The ANOVA Radial Basis Kernel can be applied to several regression issues. By focusing on every conceivable subset up to a specific size containing variables, the Anova decomposition concept was transformed into kernels.

$$K(X_i, X_j) = \sum_{k=1}^{n} e^{(-\sigma(X_i^k - X_y^k)^2)^p} \tag{45}$$

In this case, the Euclidean distance between X_i and X_j is represented by $X_i - X_j$.The variance and our hyperparameter are both "σ". RBF kernels are the most often used and generic type of kernelization because of their similarity to the Gaussian distribution. The RBF kernel function calculates the similarity, or proximity, between two points, X_i and X_j.

$$K(X_i, X_j) = e^{-\left(\frac{\|X_i - X_j\|^2}{2\sigma^2}\right)} \tag{46}$$

where σ is used to define standard deviation and can be derived by using the given formula:

$$\sigma = \sqrt{\frac{\sum_{i=0}^{n} (X_i - \mu)^2}{N}} \tag{47}$$

$\sum$ = Sum of $(X_i - \mu)^2$, X_i = each value of X, μ = mean, and N = Number of values.

A multi-class classification is divided into binary classification problems for each pair of classes in the One vs One (OVO) strategy. Our target feature has four categories including SAX, PXTG1, PXTG4, and PXTG12. To construct the classifiers, an attempt will be made to select potential combinations for each class. For each class, possible combinations are SAX vs PXTG1, SAX vs PXTG4, SAX vs PXTG12, PXTG1 vs PXTG4, PXTG1 vs PXTG12, and PXTG4 vs PXTG12. Currently, the attempt is being made to fit six distinct binary classifiers for every pair of classes. After applying these classifiers to unseen data, the class with the highest number of "+1" predictions is selected.

Algorithm 3 SVM_ModelGenerator ($\boldsymbol{\eta}$, **ep**, **D**)

1. Initialize W_i^0, W_i^1, W_i^2, $i = 1, 2, .., 6$
2. **X** = **D**[Size, Core]
3. **Y** = **D**[Efficient_Algo]
4. FOR **e** =1 to **ep**
   i. Calculate n· (n-1)/2 binary classifiers applying (35). //where n is the number of classes in the target feature and n =4.
   ii. Calculate loss.
   iii. Compute gradients for weights and biases.
   iv. Update W_i^0, W_i^1, W_i^2.
   END FOR
5. Return SVM_Model

SVM_ModelGenerator (Algorithm 3) receives learning rate ($\boldsymbol{\eta}$), number of epochs (**ep**), and dataset (**D**) from Algorithm 1. Step 1 initializes W^0, W^1, and, W^2 with random values. W^1 and W^2 represent weights and W^0 represents biases. Steps 2 and 3 extract the input features and target

feature in X and Y respectively. Step 4 consists of four sub-steps. Sub-step 1 applies OVO strategy for multi-class classification problems. It then computes six binary classifiers for each pair of four classes in the target feature. Sub-step 2 calculates loss to measure the efficiency of model. Sub-step 3 computes gradients against the current weights and biases. Sub-step 4 updates weights and biases to reduce loss. Step 4 is repeated up to the number of epochs to improve the performance of the SVM model. The final step returns the classification model to Algorithm 1. Consequently, the result of the classification phase is produced in the form of ANN and SVM classification models as given in Figure 6.

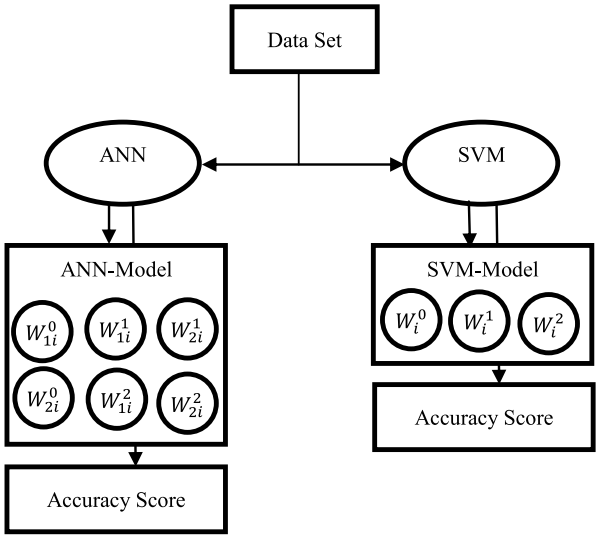


FIGURE 6. Classification using ANN and SVM.

C. PERFORMANCE EVALUATION

The performance evaluation phase of the XPOHML framework is used to compute the performance of ANN and SVM classifiers by applying accuracy. Accuracy is the most widely used metrics in machine learning to assess the performance of classifier [58]. It calculates the percentage of correctly classified instances (true positives and true negatives) and is written as:

$$Accuracy = (TP + TN)/(TP + TN + FP + FN) \quad (48)$$

False Positive (FP) is, when the model predicted an algorithm as efficient and actually it was not the case. False Negative (FN) is, when the model predicted an algorithm as not efficient and actually it was. True Positive (TP) is, when the model predicted an algorithm as efficient and actually it was. True Negative (TN) is, when the model predicted an algorithm as not efficient and actually it was not the case.

D. PARSER CODE GENERATION

The final module of the XPOHML framework generates the code of the most efficient parsing model against particular file size and the number of cores by calling the ***ParserCodeGenerator*** module (Algorithm 4), as given below:

ParserCodeGenerator (Algorithm 4) receives an XML file size and the number of cores. Step 1 calls ANN_Model to predict an efficient XML parser against a particular file size and number of cores entered by the user. The next step checks the accuracy of ANN by applying the classification metrics. Step 3 calls SVM_Model to predict an efficient XML parser against a user-entered particular file size and number of cores. Step 4 applies the classification metrics to check the accuracy of SVM. Step 5 compares the accuracy of the ANN model and SVM model. The code of the most effective parsing model against XML file and core is finally called.

Algorithm 4 ParserCodeGenerator (Size, Core)

```
1. Efficient_Algo_ANN = ANN_Model(Size, Core)
2. ANN_Score = Accuracy()
3. Efficient_Algo_SVM = SVM_Model(Size,Core)
4. SVM_Score = Accuracy()
5. IF ANN_Score>SVM_Score
        Call Efficient_Algo_ANN(Size, Core)
   ELSE
        Call Efficient_Algo_SVM(Size, Core)
   END IF-ELSE
```

IV. EXPERIMENTS AND RESULTS

Our experiments have been conducted using the novel proposed framework XPOHML. The XPOHML framework utilizes XML files of size 1, 2, 3, 4, 5, 10, 15, 20, 25, 50, 75, 100, and 125 MB for training. The XML profiling has been performed for each XML file by applying 1, 2, 4, 6, 8, 10, 12, 14, and 16 cores. The performance results after XML profiling for 75, 100, and 125 MB file sizes are given in Fig. 7-10. The architecture used for XML profiling of different algorithms is given in TABLE 1.

TABLE 1. Architecture used for XML profiling.

Processor	RAM	OS
Intel Core i7, 3.40 GHz, 64-bit	16 GB	Ubuntu 20.04.6

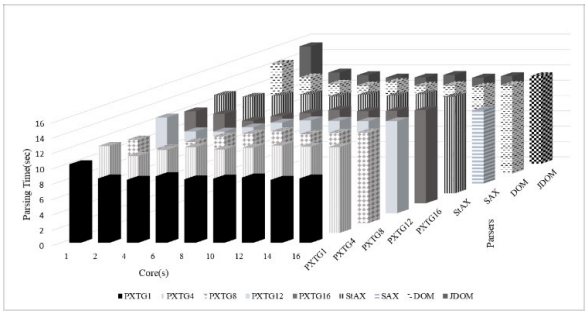


FIGURE 7. Profiling performance of different algorithms on 75 MB file size.

The XML profiling phase produces a dataset containing the file size, number of cores, and the performance of used algorithms. The XPOHML framework performs pre-processing

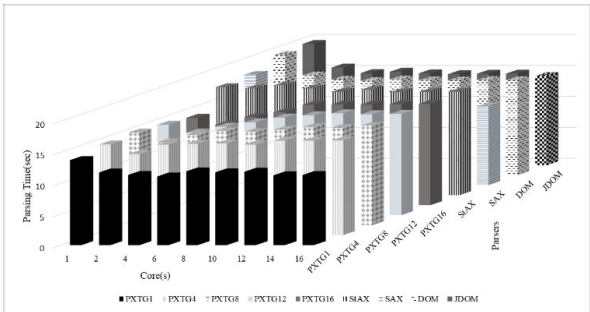


FIGURE 8. Profiling performance of different algorithms on 100 MB file size.

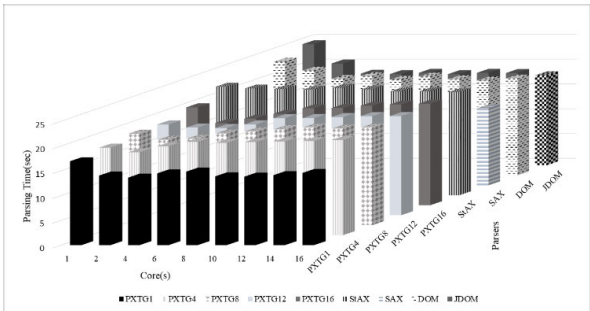


FIGURE 9. Profiling performance of different algorithms on 125 MB file size.

on the dataset by measuring the best execution time for each file size and core in Algorithm 1 (step 3). The target feature of the dataset is replaced by the name of the most efficient algorithm in terms of performance (parsing time). The input features of the finalized dataset are "Size" and "Core", while the target feature is "Efficient_Algo". The XPOHML framework then applies ANN and SVM techniques to this dataset and creates classification models. The models are tested using several XML files of varying sizes 30 MB, 60 MB, and 120 MB while executing code on 2, 4, 6, and 12 cores as given in Figures 10-12. The next phase of XPOHML evaluates the performance of both models through different classification metrics as reported in TABLE 2. Finally, the parser code of the most efficient algorithm is generated as given in Algorithm 4 (step 5) by the XPOHML framework.

TABLE 2. Performance evaluation on testing data.

Classification Metrics	ANN	SVM
Accuracy	0.88	0.90

Figure 13 shows the average performance time of the XPOHML framework across the SAX, StAX, DOM, JDOM, PXTG4, PXTG8, PXTG12, and PXTG16 parsing models. The results of parsing XML files, as shown in Figure 13 demonstrate that the code produced by the proposed framework performs better than other parsing models, yielding average gain over PXTG4, PXTG8, PXTG12, PXTG16, StAX, SAX, DOM, and JDOM of 28.52%, 31.43%, 35.52%, 32.78%, 49.2%, 9.7%, 39.3%, and 34.06%, respectively.

TABLE 3. Improvement percentage of XPOHML over other algorithms for 30 MB file size.

Cores	2	6	12	16
PXTG4	10.34	24.18	27.95	29.89
PXTG8	13.18	27.99	33.97	35.05
PXTG12	12.66	28.26	37.26	37.50
PXTG16	12.66	30.43	37.81	35.60
StAX	48.58	50.54	47.12	44.84
SAX	08.01	11.14	12.33	13.59
DOM	32.04	31.25	35.07	32.34
JDOM	32.04	31.52	32.33	33.70

TABLE 4. Improvement percentage of XPOHML over other algorithms for 60 MB file size.

Cores	2	6	12	16
PXTG4	14.81	25.3	32.41	31.93
PXTG8	10.85	26.36	38.27	37.54
PXTG12	12.61	27.26	38.27	38.32
PXTG16	09.53	29.07	40.90	44.39
StAX	39.30	43.22	47.99	48.29
SAX	09.38	09.19	10.80	12.62
DOM	34.46	31.33	35.96	37.54
JDOM	30.79	30.27	32.87	36.45

TABLE 5. Improvement percentage of XPOHML over other algorithms for 120 MB file size.

Cores	2	6	12	16
PXTG4	20.39	31.91	39.32	33.75
PXTG8	24.26	29.64	45.64	37.68
PXTG12	27.75	28.25	46.18	39.65
PXTG16	19.33	29.05	48.19	43.15
StAX	56.94	50.11	54.97	45.41
SAX	12.05	05.94	10.87	06.92
DOM	53.68	40.72	42.95	36.22
JDOM	49.36	28.76	37.39	26.17

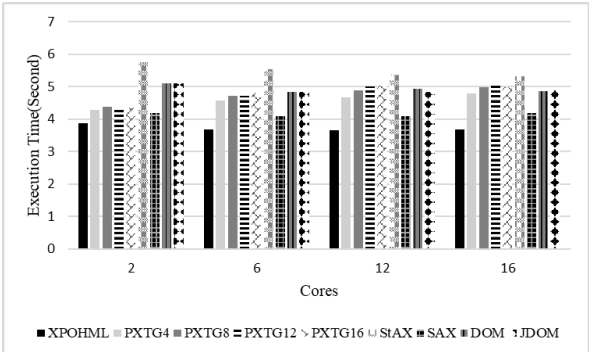


FIGURE 10. Results corresponding to 30 MB file size.

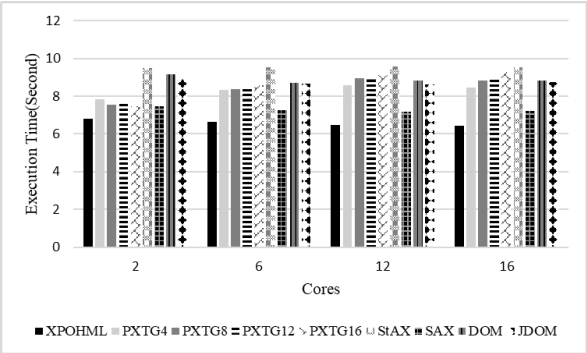


FIGURE 11. Results corresponding to 60 MB file size.

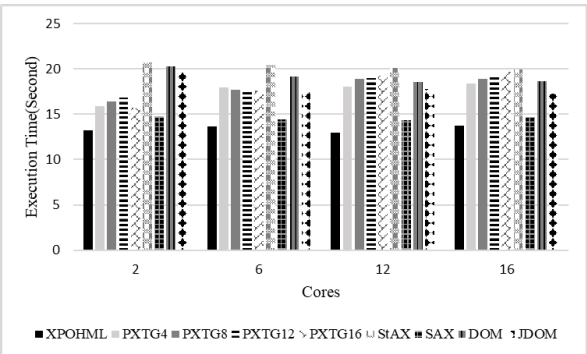


FIGURE 12. Results corresponding to 120 MB file size.

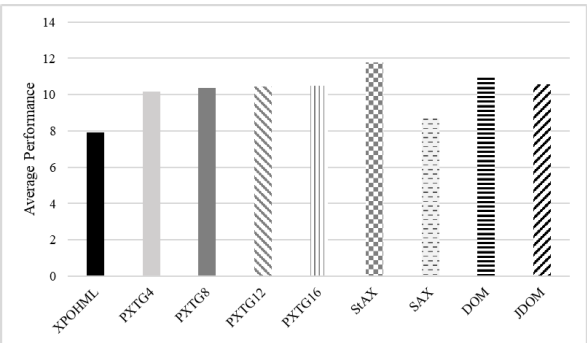


FIGURE 13. Average improvement performance of XPOHML over other parsing models.

V. CONCLUSION

For efficient XML parsing, the hybrid machine learning approach proposed in this paper is implemented using the XPOHML framework that performs analysis and prediction of different parsers using profiling, classification, performance evaluation, and code generation phases. The XML profiling phase of the XPOHML framework produces a dataset by measuring the performance of PXTG, SAX, StAX, DOM, and JDOM parsing models on different cores by using multiple file sizes. The classification phase produces the classifiers by applying ANN and SVM techniques to identify the appropriate parsing model. The performance evaluation phase of XPOHML assesses the efficacy of parsing models by implementing accuracy metrics to compare performance outcomes. Additionally, based on previous outcomes, the code generation phase produces an efficient code for parsing XML files. The parser generated by the XPOHML framework outperforms other parsing models in terms of parsing time.

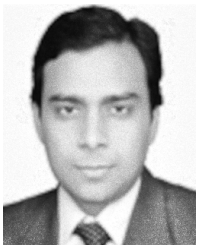
REFERENCES

[1] C. Li and T. W. Ling, "QED: A novel quaternary encoding to completely avoid re-labeling in XML updates," in *Proc. 14th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2005, pp. 501–508.
[2] T. Lam, J. J. Ding, and J.-C. Liu, "XML document parsing: Operational and performance characteristics," *Computer*, vol. 41, no. 9, pp. 30–37, Sep. 2008.
[3] I. Ahmad, S. Patil, and S. R. Sarangi, "HPXA: A highly parallel XML parser," in *Proc. Design, Autom. Test Eur. Conf. Exhibition (DATE)*, Mar. 2018, pp. 249–252.
[4] Z. Dai, N. Ni, and J. Zhu, "A 1 cycle-per-byte XML parsing accelerator," in *Proc. 18th Annu. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Feb. 2010, pp. 199–208.
[5] I. Kaplan, *Processing XML With the XML Pull Parser*. USA: BearCave, 2006. [Online]. Available: http://bearcave.com/software/java/xml/xmlpull.html
[6] M. L. Noga, S. Schott, and W. Löwe, "Lazy XML processing," in *Proc. ACM Symp. Document Eng.*, Nov. 2002, pp. 88–94.
[7] W. M. Lowe, M. L. Noga, and T. S. Gaul, "Foundations of fast communication via XML," *Ann. Softw. Eng.*, vol. 13, nos. 1–4, pp. 357–379, 2002.
[8] Z. Pan, X. Jiang, J. Wu, and X. Li, "Hybrid XML parser based on software and hardware co-design," in *Proc. IEEE 27th Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, Apr. 2019, p. 325.
[9] S. Skansi, *Introduction to Deep Learning: From Logical Calculus to Artificial Intelligence*. Springer, 2018.
[10] Y. Lu Murphey, M. A. Masrur, Z. Chen, and B. Zhang, "Model-based fault diagnosis in electric drives using machine learning," *IEEE/ASME Trans. Mechatronics*, vol. 11, no. 3, pp. 290–303, Jun. 2006.
[11] B. Li, M.-Y. Chow, Y. Tipsuwan, and J. C. Hung, "Neural-network-based motor rolling bearing fault diagnosis," *IEEE Trans. Ind. Electron.*, vol. 47, no. 5, pp. 1060–1069, Oct. 2000.
[12] B.-R. Lin and R. G. Hoft, "Power electronics inverter control with neural networks," in *Proc. 8th Annu. Appl. Power Electron. Conf. Expo.*, Mar. 1993, pp. 128–134.
[13] B. K. Bose, "Neural network applications in power electronics and motor drives—An introduction and perspective," *IEEE Trans. Ind. Electron.*, vol. 54, no. 1, pp. 14–33, Feb. 2007.
[14] G. Tsekouras, S. Kiartzis, A. G. Kladas, and J. A. Tegopoulos, "Neural network approach compared to sensitivity analysis based on finite element technique for optimization of permanent magnet generators," *IEEE Trans. Magn.*, vol. 37, no. 5, pp. 3618–3621, Sep. 2001.
[15] A. Sadeghian and J. D. Lavers, "Implementation of knowledge-based system for iron core inductor design," *IEEE Trans. Magn.*, vol. 40, no. 6, pp. 3495–3504, Nov. 2004.
[16] I. H. Sarker, M. H. Furhad, and R. Nowrozy, "AI-driven cybersecurity: An overview, security intelligence modeling and research directions," *Social Netw. Comput. Sci.*, vol. 2, no. 3, pp. 1–18, May 2021.
[17] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *Social Netw. Comput. Sci.*, vol. 2, no. 3, p. 160, May 2021.
[18] A. T. M. Mohri, *Foundations of Machine Learning*, 2nd ed., Cambridge, MA, USA: MIT Press, Dec. 25, 2018.
[19] R. Atienza, *Advanced Deep Learning With TensorFlow 2 and Keras: Apply DL, GANs, VAEs, Deep RL, Unsupervised Learning, Object Detection and Segmentation, and More*. Birmingham, U.K.: Packt Publishing Ltd, 2020.
[20] F. Chollet, *Deep Learning With Python*. New York, NY, USA: Simon and Schuster, 2021.
[21] J. Robie, *What is the Document Object Model?* USA: W3C, 2004. [Online]. Available: https://www.w3.org/TR/2004/REC-DOM-Level-3-Core-20040407/
[22] D. Megginson, *Simple API for XML*. SAXProject.org, 2004. [Online]. Available: http://www.saxproject.org/sax1-roadmap.html
[23] M. Ali and M. A. Khan, "Performance enhancement of XML parsing using regression and parallelism," *Comput. Syst. Sci. Eng.*, vol. 48, no. 2, pp. 287–303, 2024, doi: 10.32604/CSSE.2023.043010.
[24] J. Zhang, *Simplify XML Processing With VTD-XML*. USA: InfoWorld, 2006. [Online]. Available: https://www.infoworld.com/article/2157634/simplify-xml-processing-with-vtd-xml.html
[25] K. T. Schneider, *Efficient XML Interchange (EXI) Format 1.0*, 2nd ed., USA: W3C, 2014. [Online]. Available: https://www.w3.org/TR/exi/

[26] M.-L. Chai, S. Song, and N.-N. Li, "A review of some main improved models for neural network forecasting on time series," in *Proc. IEEE Intell. Vehicles Symp.*, Jun. 2005, pp. 866–868.
[27] J. W. Woolley, P. K. Agarwal, and J. Baker, "Modeling and prediction of chaotic systems with artificial neural networks," *Int. J. Numer. Methods Fluids*, vol. 63, no. 8, pp. 989–1004, Jul. 2010.
[28] B. Schölkopf, A. J. Smola, R. C. Williamson, and P. L. Bartlett, "New support vector algorithms," *Neural Comput.*, vol. 12, no. 5, pp. 1207–1245, May 2000.
[29] G. Zhu, G. Zhong, and H. Deng, "A method to improve the robustness of grasping pattern recognition for prosthetic hands," in *Proc. 3rd Int. Conf. Syst. Informat. (ICSAI)*, Nov. 2016, pp. 332–337.
[30] D. Gong, Z. Li, W. Huang, X. Li, and D. Tao, "Heterogeneous face recognition: A common encoding feature discriminant approach," *IEEE Trans. Image Process.*, vol. 26, no. 5, pp. 2079–2089, May 2017.
[31] W. YuKun, Z. Yong, W. JieSheng, and C. Ying, "Research of flotation dosing control method based on SVM and image processing technology," in *Proc. 31st Chin. Control Conf.*, Jul. 2012, pp. 7067–7070.
[32] M. Marufo da Silva, D. A. Evin, and S. Verrastro, "Speaker-independent embedded speech recognition using hidden Markov models," in *Proc. IEEE CACIDI IEEE Conf. Comput. Sci.*, Nov. 2016, pp. 1–6.
[33] S.-S. Lee, S.-J. Jang, J. Kim, and B. Choi, "A hardware architecture of face detection for human–robot interaction and its implementation," in *Proc. IEEE Int. Conf. Consum. Electron.-Asia (ICCE-Asia)*, Oct. 2016, pp. 1–2.
[34] M. Khishe and M. R. Mosavi, "Improved whale trainer for sonar datasets classification using neural network," *Appl. Acoust.*, vol. 154, pp. 176–192, Nov. 2019.
[35] M. R. Mosavi, M. Khishe, and M. Akbarisani, "Neural network trained by biogeography-based optimizer with chaos for sonar data set classification," *Wireless Pers. Commun.*, vol. 95, no. 4, pp. 4623–4642, Aug. 2017.
[36] M. Kaveh, M. Khishe, and M. R. Mosavi, "Design and implementation of a neighborhood search biogeography-based optimization trainer for classifying sonar dataset using multi-layer perceptron neural network," *Anal. Integr. Circuits Signal Process.*, vol. 100, no. 2, pp. 405–428, Aug. 2019.
[37] M. R. Mosavi and M. Khishe, "Training a feed-forward neural network using particle swarm optimizer with autonomous groups for sonar target classification," *J. Circuits, Syst. Comput.*, vol. 26, no. 11, Nov. 2017, Art. no. 1750185.
[38] M. Khishe and H. Mohammadi, "Passive sonar target classification using multi-layer perceptron trained by salp swarm algorithm," *Ocean Eng.*, vol. 181, pp. 98–108, Jun. 2019.
[39] M. Khishe and A. Safari, "Classification of sonar targets using an MLP neural network trained by dragonfly algorithm," *Wireless Pers. Commun.*, vol. 108, no. 4, pp. 2241–2260, Oct. 2019.
[40] M. Khishe and M. R. Mosavi, "Classification of underwater acoustical dataset using neural network trained by chimp optimization algorithm," *Appl. Acoust.*, vol. 157, Jan. 2020, Art. no. 107005.
[41] W. Qiao, M. Khishe, and S. Ravakhah, "Underwater targets classification using local wavelet acoustic pattern and multi-layer perceptron neural network optimized by modified whale optimization algorithm," *Ocean Eng.*, vol. 219, Jan. 2021, Art. no. 108415.
[42] T. Hu, M. Khishe, M. Mohammadi, G.-R. Parvizi, S. H. Taher Karim, and T. A. Rashid, "Rea-time COVID-19 diagnosis from X-ray images using deep CNN and extreme learning machines stabilized by chimp optimization algorithm," *Biomed. Signal Process. Control*, vol. 68, Jul. 2021, Art. no. 102764.
[43] C. Wu, M. Khishe, M. Mohammadi, S. H. Taher Karim, and T. A. Rashid, "Evolving deep convolutional neutral network by hybrid sine-cosine and extreme learning machine for real-time COVID19 diagnosis from X-ray images," *Soft Comput.*, vol. 27, no. 6, pp. 3307–3326, 2023.
[44] H. M. Ahmed, B. A. B. Youssef, A. S. Elkorany, Z. F. Elsharkawy, A. A. Saleeb, and F. A. El-Samie, "Hybridized classification approach for magnetic resonance brain images using gray wolf optimizer and support vector machine," *Multimedia Tools Appl.*, vol. 78, no. 19, pp. 27983–28002, Oct. 2019.
[45] A. S. Elkorany and Z. F. Elsharkawy, "Automated optimized classification techniques for magnetic resonance brain images," *Multimedia Tools Appl.*, vol. 79, nos. 37–38, pp. 27791–27814, Oct. 2020.
[46] T. Mu and A. K. Nandi, "Detection of breast cancer using v-SVM and RBF networks with self organized selection of centers," in *Proc. 3rd IEE Int. Seminar Med. Appl. Signal Process.*, Nov. 2005, pp. 47–52.
[47] M. Nilashi, O. Ibrahim, H. Ahmadi, and L. Shahmoradi, "A knowledge-based system for breast cancer classification using fuzzy logic method," *Telematics Informat.*, vol. 34, no. 4, pp. 133–144, Jul. 2017.
[48] M. Kumari and V. Singh, "Breast cancer prediction system," *Proc. Comput. Sci.*, vol. 132, pp. 371–376, May 2018.
[49] V. Vapnik, "Pattern recognition using generalized portrait method," *Autom. Remote Control*, vol. 24, pp. 774–780, Jan. 1963.
[50] B. E. Boser, I. M. Guyon, and V. N. Vapnik, "A training algorithm for optimal margin classifiers," in *Proc. 5th Annu. ACM Workshop Comput. Learn. Theory*, 1992, pp. 144–152.
[51] S. T. A. K. Koutroumbas, *Pattern Recognition*, 4th ed., New York, NY, USA: Academic, 2009, p. 10.
[52] (2004). *ENVI User's Guide*. [Online]. Available: https://www.nv5geospatialsoftware.com/portals/0/pdfs/enviex/envi_ex_user_guide.pdf
[53] S. Van der Linden, A. Rabe, M. Held, B. Jakimow, P. Leitão, A. Okujeni, M. Schwieder, S. Suess, and P. Hostert, "The EnMAP-box—A toolbox and application programming interface for EnMAP data processing," *Remote Sens.*, vol. 7, no. 9, pp. 11249–11266, Sep. 2015.
[54] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995.
[55] T. Phienthrakul, B. Kijsirikul, H. Takamura, and M. Okumura, "Sentiment classification with SVM and MKL," in *Proc. Neural Inf. Proc., 16th Int. Conf.*, 2009, pp. 583–591.
[56] C. Bishop, *Pattern Recognition and Machine Learning*, vol. 2. Cham, Switzerland: Springer, 2006, pp. 531–537.
[57] Y. Jiao, J. J. Hall, and Y. T. Morton, "Automatic equatorial GPS amplitude scintillation detection using a machine learning algorithm," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 53, no. 1, pp. 405–418, Feb. 2017.
[58] P. Peduzzi, J. Concato, A. R. Feinstein, and T. R. Holford, "Importance of events per independent variable in proportional hazards regression analysis II. Accuracy and precision of regression estimates," *J. Clin. Epidemiology*, vol. 48, no. 12, pp. 1503–1510, Dec. 1995.

MUHAMMAD ALI received the M.S. degree in computer science from Bahauddin Zakariya University, Multan, Pakistan, in 2012, where he is currently pursuing the Ph.D. degree in computer science with the Department of Computer Science.

His research interests include XML parsing, high-performance computing, and machine learning. He has also a vast practical experience in professional programming with various languages.



MINHAJ AHMAD KHAN received the M.S. and Ph.D. degrees in computer science from the University of Versailles Saint-Quentin-en-Yvelines, France, in 2005 and 2008, respectively.

He also holds a Postdoctoral Researcher with the University of Bordeaux-I, France. He is currently associated with the Department of Computer Science, Bahauddin Zakariya University, Multan. He has several publications in prestigious journals and conferences on these topics. His research interests include the Internet of Things, cloud computing, high-performance computing, task scheduling, and dynamic optimization of code.

RAIHAN UR RASOOL is currently a Senior Architect with IBM Technologies, Australia, and a Fulbright Alumnus with The University of Chicago, USA. His work comprising more than 80 articles, is published in various international conferences and journals of high repute. His research interests include large-scale distributed systems and hybrid clouds, security, and computer architecture.